

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
Институт математики и информационных систем
Факультет автоматизации и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №1
по дисциплине
«Управление данными»
«Основы DDL-запросов в PostgreSQL .»

Выполнил студент гр. ИВТб-2303-06-00 _____/Перевозчиков Д.А./

Проверил xxxxxxxx _____/Х Х. Х./

Киров
2026

1 Цели лабораторной работы:

- познакомиться со схемами, пользователями и ролями в PostgreSQL;
- познакомиться с типами данных в PostgreSQL;
- освоить основные варианты DDL-запросов в PostgreSQL;
- закрепить знания по проектированию структуры реляционной БД;
- создать рабочий материал для следующих лабораторных работ.

2 Задание:

1. Разработать структуру базы данных на любую выбранную тему.

Структура должна отвечать следующим условиям:

- должно быть не меньше пяти таблиц;
- хотя бы одна таблица должна содержать колонку с числовыми данными;
- структура БД не должна быть связь много-ко-многом;

2. Создать нового пользователя и пустую БД. Подключиться к созданной БД.

3. Написать и выполнить SQL-скрипт, создающий таблицы согласно разработанной структуре БД. Созданный в п.2 пользователь должен иметь все права на созданные объекты. В этом же скрипте должны создаваться нужные ограничения и индексы:

- обязательно должны быть созданы внешние ключи для поддержания ссылочной целостности;

- желательно должны быть проставлены ограничения и уникальные индексы для поддержания консистентности данных (например, если в таблице есть столбец с температурой воздуха за окном, стоит ограничить возможный промежуток значений);
- желательно должны быть проставлены индексы для производительности там, где они могут помочь (из расчета, что созданная БД будет хранить достаточно большое количество данных;

3 Решение:

3.1 Задание 1.

Для выполнения задания смоделируем такие условия: Необходимо спроектировать базу данных для хранения информации о доступных ресурсах гоночного клуба. Проектируемая система должна выполнять следующие действия:

- Редактировать автопарк - набор гоночных автомобилей, гоночный автомобиль можно добавить, редактировать информацию, отправить в утилизацию. Автомобилем в утилизации клиенты воспользоваться не могут.
- Гоночный автомобиль состоит из заголовка, фото, описания, кол-ва лошадиных сил, года выпуска, цвета, наименования подвески.
- Редактировать тренерский состав - добавлять, удалять, редактировать информацию о тренере.
- Информация о тренере включает в себя ФИО, года рождения, номер счёта в банке для начисления заработной платы, фото тренера, список достижений тренера, список автомобилей, на которых тренер может тренировать. А также любимый автомобиль тренера.
- Список достижений тренера состоит из элементов, каждый из которых включает в себя наименование соревнования, дату проведения соревнования и занятое место.

Учитывая все вышеперечисленные требования, разработаем схему базы данных гоночного клуба.

ER-диаграмма для задания 1 представлена на Рисунке 1.

Исходный код представлен в Приложении А1.

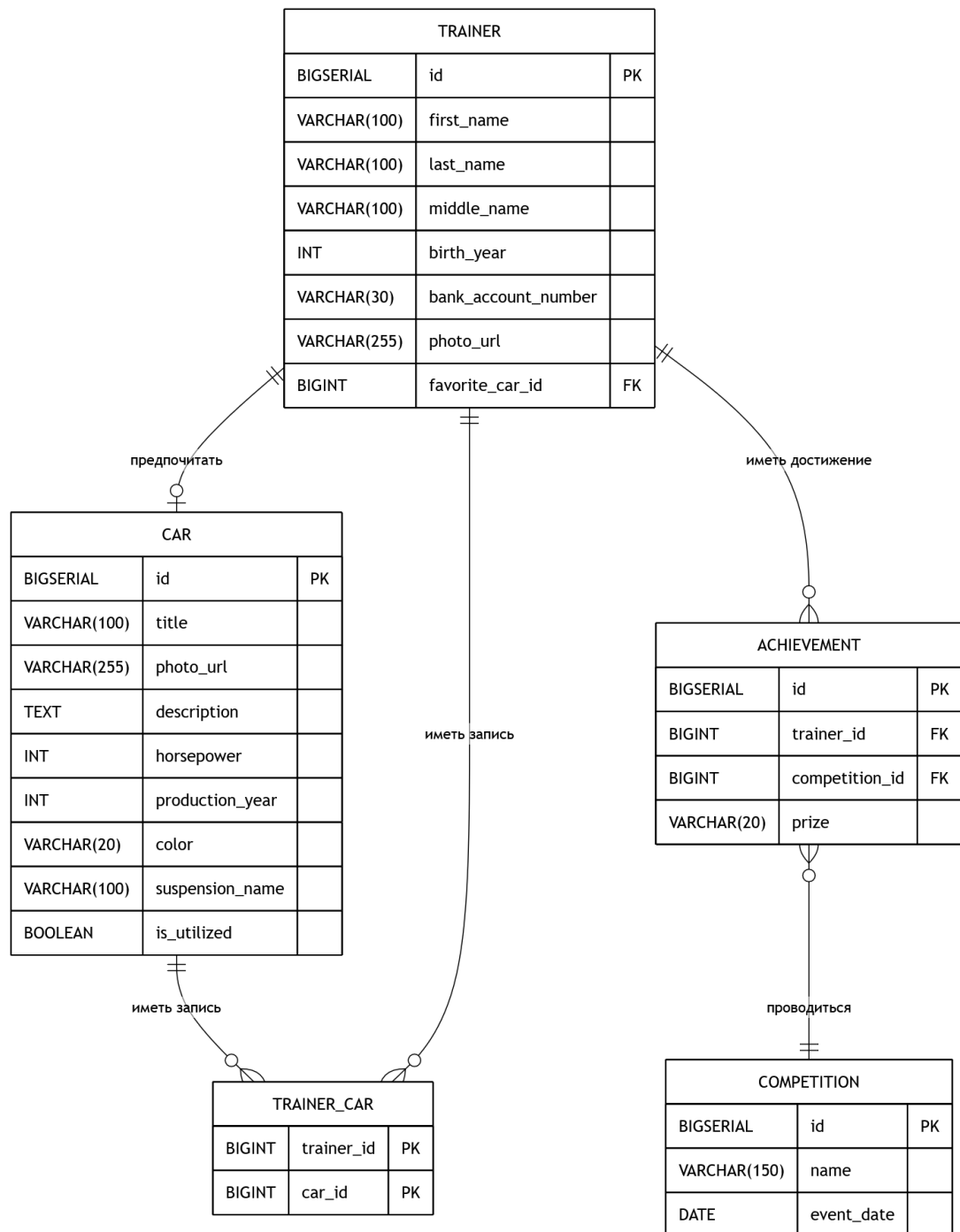


Рисунок 1 - ER Диаграмма задания 1

3.2 Задание 2.

С использованием консольной утилиты **psql** был создан пользователь базы данных roiten. От его имени была создана база для решения задачи от автоклуба.

```
PS C:\Users\roiten> psql -U postgres
Пароль пользователя postgres:

psql (18.6)
Введите "help", чтобы получить справку.

postgres=# CREATE USER roiten PASSWORD '12345Q';
CREATE ROLE
postgres=# CREATE DATABASE autoclub OWNER roiten;
CREATE DATABASE
postgres=# \c autoclub
Вы подключены к базе данных "autoclub" как пользователь "postgres".
autoclub=# \q
PS C:\Users\roiten> psql -U roiten -d autoclub
Пароль пользователя roiten:

psql (18.6)
Введите "help", чтобы получить справку.

autoclub=> |
```

Рисунок 2 - Создание пользователя и базы данных.

3.3 Задание 3.

В соответствии с разработанной структурой базы данных были написаны SQL-запросы, создающие все таблицы, внешние ключи, ограничения и индексы. Скрипт выполнен от имени пользователя, созданного в п.2. Полный текст SQL-скрипта со всеми ограничениями и индексами приведён в Приложении А1.

Вывод:

В ходе выполнения лабораторной работы были освоены схемы, пользователи и роли в PostgreSQL, типы данных, основные варианты DDL-запросов, а также закреплены знания по проектированию структуры реляционной БД. Создан рабочий материал для следующих лабораторных работ.

Приложение A1

Исходный код запросов для задания 3:

-- Таблица гоночных автомобилей

```
CREATE TABLE IF NOT EXISTS car (
    id BIGSERIAL PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    photo_url VARCHAR(255),
    description TEXT,
    horsepower INT NOT NULL CHECK (horsepower > 0),
    production_year INT NOT NULL CHECK (production_year > 1900),
    color VARCHAR(20) NOT NULL,
    suspension_name VARCHAR(100) NOT NULL,
    is_utilized BOOLEAN NOT NULL DEFAULT FALSE
);
```

-- Индекс для поиска по названию

```
CREATE INDEX ON car(title);
```

-- Таблица тренеров

```
CREATE TABLE IF NOT EXISTS trainer (
    id BIGSERIAL PRIMARY KEY,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    middle_name VARCHAR(100),
    birth_year INT NOT NULL,
    bank_account_number VARCHAR(30) UNIQUE NOT NULL,
    photo_url VARCHAR(255),
    favorite_car_id BIGINT REFERENCES car(id) ON DELETE SET NULL,

    CHECK (birth_year > 1900)
);
```

-- Индекс для поиска по ФИО

```
CREATE INDEX ON trainer(last_name, first_name, middle_name);
```

-- Промежуточная таблица: тренер - автомобили, на которых может тренировать

```
CREATE TABLE IF NOT EXISTS trainer_car (  
    trainer_id BIGINT NOT NULL REFERENCES trainer(id) ON DELETE CASCADE,  
    car_id BIGINT NOT NULL REFERENCES car(id) ON DELETE CASCADE,  
    PRIMARY KEY (trainer_id, car_id)  
);
```

-- Индекс для поиска по id авто

```
CREATE INDEX ON trainer_car(car_id);
```

-- Таблица соревнований

```
CREATE TABLE IF NOT EXISTS competition (  
    id BIGSERIAL PRIMARY KEY,  
    name VARCHAR(150) NOT NULL,  
    event_date DATE NOT NULL,  
    CHECK (event_date > DATE '1900-01-01')  
);
```

-- Таблица достижений тренера

```
CREATE TABLE IF NOT EXISTS achievement (  
    id BIGSERIAL PRIMARY KEY,  
    trainer_id BIGINT NOT NULL REFERENCES trainer(id) ON DELETE CASCADE,  
    competition_id BIGINT NOT NULL REFERENCES competition(id) ON DELETE CASCADE,  
    prize VARCHAR(20) NOT NULL  
);
```

-- Индекс для поиска достижений по id тренера

```
CREATE INDEX ON achievement(trainer_id);
```